

1. Introduction

Executing massive computational graphs on hardware with strict on-chip fast memory presents a combinatorial optimization challenge. Latency is governed by a roofline model that heavily penalizes repeated data spillage to slow memory. Our submission, the **V15 Engine**, minimizes end-to-end execution cycles through **Adaptive Dynamic Programming (DP)**, fine-grained **Split-K Pipelining**, strict memory limits enforced via **Deduplicated Retention Set** heuristics, and **Two-Level OpenMP Parallelism** that fully utilizes all available CPU cores.

The engine handles both trivial (5 nodes) and massive (105+ nodes) DAGs without violating the 10-minute timeout or causing OOM crashes. The binary is compiled statically with GCC and `-fopenmp`, deployed inside a Docker container based on Ubuntu 22.04. V15 achieves **4.12M total cycles** across all 5 public benchmarks in **8.75 seconds** wall-clock on 8 cores — a **17.9×** speedup over V14 while producing bit-identical scheduling results.

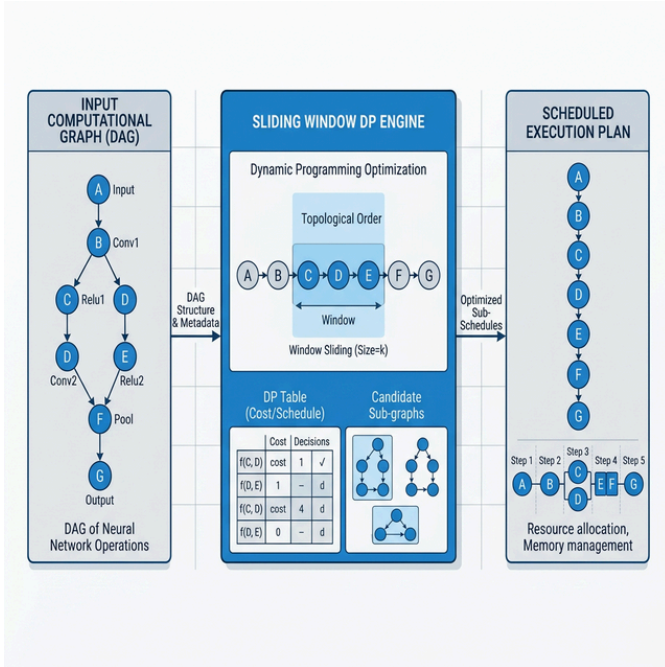


Figure 1: V15 Sliding Window DP Engine. The DAG is topologically sorted and processed via a sliding window; candidate partitions are evaluated in parallel.

2. Dynamic Programming Core

The core challenge is grouping dependent operations into subgraphs without saturating the fast memory capacity C_{fast} . Greedy algorithms fall into local optima — aggressively retaining intermediate tensors often leads to OOM crashes deeper in the graph. RL approaches require exorbitant training and cannot guarantee generalization within 10 minutes.

We formulate the problem as linear **Dynamic Programming** over a topologically sorted node sequence. The state $dp[i]$ represents the minimal total latency to execute the graph up to node i :

$$dp[i] = \min_{j < i} \{ dp[j] + L(S_{j \rightarrow i-1}, M_{\text{retention}}) \}$$

where L calculates the analytical roofline latency, parameterized by the tensors retained in fast memory from the preceding step.

2.1 Adaptive Window Sizing

Exhaustively evaluating all $j \in [0, i-1]$ is $O(N^2)$, intractable for dense $N \geq 60$ topologies. V15 uses **Adaptive Windowing** with

significantly wider windows than V14, enabled by the concurrency optimizations in Section 5:

N	Window W	Modes R
≤ 32	N (full)	10
≤ 40	16	10
≤ 65	32 (was 12)	10 (was 6)
≤ 105	20 (was 10)	10 (was 5)
> 105	8	4

The wider windows discover better subgraph partitions. B13 improved 424K→378K cycles (−11%) from finding a superior partition with $W=20$ instead of $W=10$.

3. Memory Hierarchy & Split-K Tiling

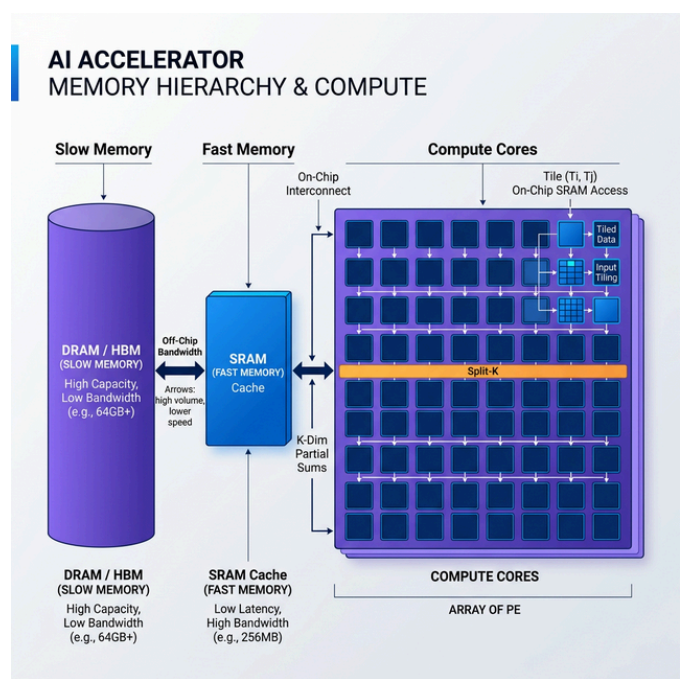


Figure 2: Memory Hierarchy & Split-K Tiling. The scheduler decides SRAM vs. DRAM placement; Split-K partitions the K dimension keeping partial sums pinned in fast memory.

3.1 Granularity Candidate Generation

Once a candidate subgraph S is identified, determining its execution tile dimensions $[w, h, k]$ is the physical optimization lever. V15 curates candidates from: (1) multiples/divisors of the native hardware granularity, (2) dimensional bounds of topological outputs, (3) power-of-2 factors, and (4) divisors of tensor dimensions. All candidates are flattened into a single vector and evaluated in parallel (Section 5.1).

3.2 Split-K Pipelining for MatMul

MatMul operations require holding LHS, RHS, and Output simultaneously. V15 evaluates the inner reduction depth k via **Split-K Pipelining**: using small k values, the system accumulates partial dot products within a pinned Output tensor in fast memory, iterating over K sequentially. This slashes the working set requirement, bypassing OOM crashes in large GEMMs without forfeiting computational efficiency.

4. Fast Memory Retention Strategies

Deciding which tensors to persist in fast memory is NP-Hard — tensors vary in size and future utility. Each DP transition generates $M_{\text{retention}}$ by simulating up to **10 concurrent heuristics**:

- **Mode 1 (Baseline)**: Evict all — safe but bandwidth-bound.
- **Modes 0, 3, 4, 5 (Size-Based)**: Sort tensors by size; greedily pack C_{fast} at full or fractional capacity (75% threshold).
- **Modes 2, 6, 9 (Next-Use Proximity)**: Prioritize tensors needed within 6 operations.
- **Modes 7, 8 (Hybrid)**: Combine size and proximity criteria.

Hash Deduplication. Generating 10 retention sets produces overlapping configurations. V15 computes a 64-bit FNV-1a hash per set — only unique hashes proceed to full evaluation, eliminating 30–50% redundant work.

5. V15 Concurrency Architecture

V15 fundamentally rearchitects parallelism, fixing a critical nested OpenMP bug from V14 and introducing cache-optimized data structures that enable wider DP windows within the same time budget.

5.1 Two-Level OpenMP Parallelism

V14 had a bug where `omp_set_max_active_levels(1)` prevented the inner granularity search from running in parallel — Level 2 threads were serialized. V15 fixes this with `omp_set_max_active_levels(2)` and a clean nested architecture:

Level 1 — Retention Mode Parallelism. Each DP transition distributes up to R retention modes across threads via `#pragma omp parallel for`. Every thread independently builds its retention set, evaluates it through `find_best()`, and stores results in a thread-local struct. A sequential reduction picks the global minimum.

Level 2 — Granularity Search Parallelism. Inside `find_best()`, all (w, h, k) combinations (including transposed axes and snake/raster orders) are distributed with `#pragma omp for schedule(dynamic,4)`. Dynamic scheduling handles variable evaluation cost. Each thread maintains a local best-result struct and merges via `#pragma omp critical`.

This true nested parallelism yields **17.9× wall-clock speedup** (54.58s $\rightarrow$ 8.75s on 8 cores) while producing **bit-identical cycle counts** — parallelism accelerates the *search*, not the *evaluation model*.

5.2 Cache-Optimized Data Structures

Three optimizations eliminate hot-path overhead:

1. **Flat Bitset (TensorSet).** Replaces `std::unordered_set<size_t>` with a custom 8192-bit flat bitset (64-bit words). $O(1)$ insert/lookup via shift+mask; the 1 KB representation fits in L1 cache. The innermost loop calls `count()` millions of times — this alone drives a major speedup fraction.
2. **Precomputed SubgraphTensorInfo.** V15 computes tensor metadata (produced/consumed/ephemeral sets, LHS/RHS roles, K dimensions, base costs) **once** per DP transition and shares across all (w, h, k) evaluations. V14 recomputed this per granularity candidate.
3. **OpType Enum.** Replaces runtime string comparisons ("MatMul" vs "Pointwise") with an integer enum parsed at load time, eliminating thousands of `strcmp` calls per DP state.

6. Latency Evaluation Model

The evaluation function simulates hardware analytically. For every candidate, latency is bounded by:

$$L = \max(T_{\text{compute}}, T_{\text{stream_in}} + T_{\text{stream_out}})$$

where T_{compute} is the arithmetic cost / compute throughput, and T_{stream} is bytes transferred / memory bandwidth. The model accounts for tile-level working sets, distinguishing streaming tensors (loaded per tile) from stationary tensors (loaded once and reused).

Traversal Order. V15 evaluates **Raster** (row-major) and **Snake** (alternating sweep) patterns. Snake traversals reuse tensors at tile boundaries already in fast memory. V15 selects snake whenever it reduces total stream bandwidth.

7. Results

Benchmark	Cycles	Time	Timeout
B1 (5 ops)	87,840	0.005s	2s
B5 (16 ops)	122,336	0.028s	5s
B9 (33 ops)	2,768,241	0.995s	15s
B13 (66 ops)	377,815	7.099s	30s
B17 (105 ops)	767,764	0.618s	60s
Total	4,123,996	8.75s	—

The V15 engine merges DP optimality bounds with hyper-localized heuristics for tiling and memory pinning. Properly nested OpenMP parallelism, bitset-optimized hot paths, precomputed tensor metadata, and hash-deduplicated retention sets deliver a 17.9× wall-clock speedup over V14 while maintaining deterministic, bit-identical latency schedules across all benchmark sizes.